

Universität Stuttgart
Master of Science

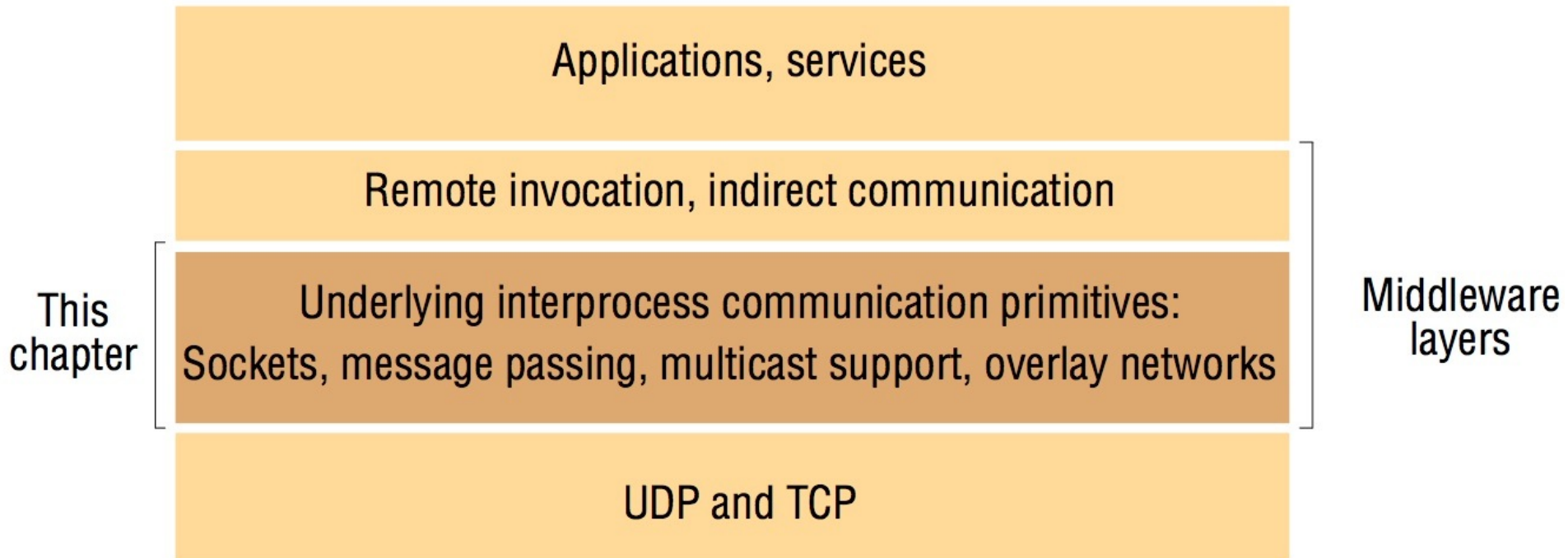
Interprocess Communication

Marco Aiello

Interprocess communication: middleware

- Middleware provides API to enable the communication of processes as a result of application requests relying on the transport layer

Figure 4.1
Middleware layers



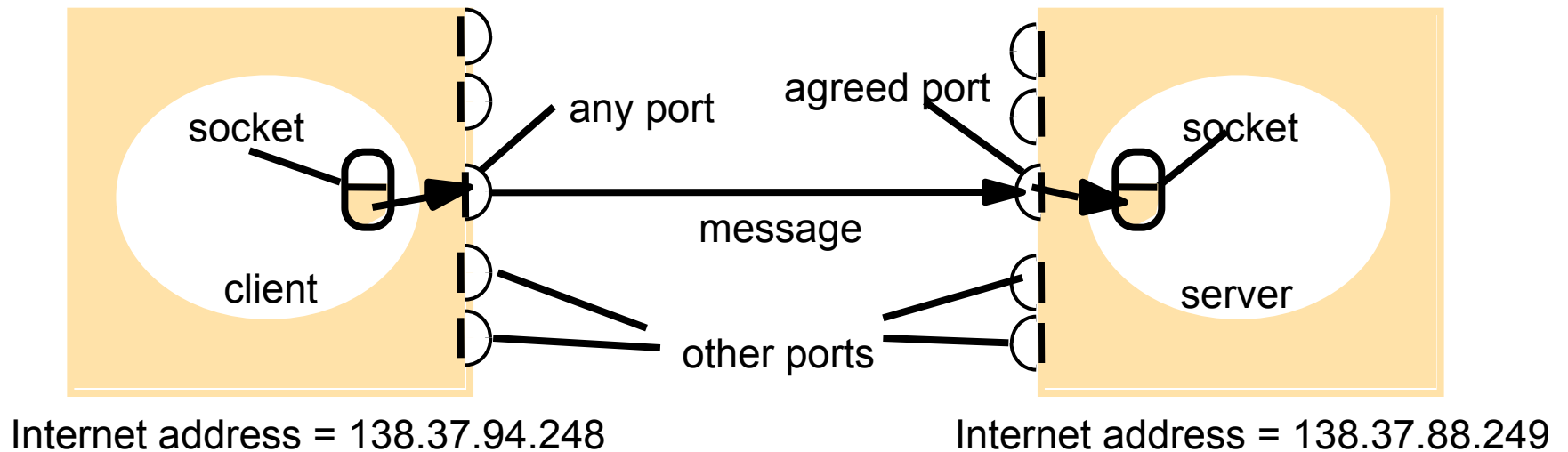
Interprocess communication

- **Message passing:** *send* and *receive* of messages <destination, content>
- **Synchronous vs. asynchronous communication:** sync. implies blocking send and receive (multithreading allows the use of blocking operations, otherwise processes would hang)
- **Destination:** network address + port (a destination within a host which identifies a receiving process). Ports are unique.

Sockets

- A *socket* is the abstraction of a connection from the process' perspective
- **E.g., on UNIX a pair of client-server processes each establish their own socket:**
- *client side:*
 1. Create a socket with the `socket()` system call
 2. Connect the socket to the address of the server using the `connect()` system call
 3. Send and receive data via UDP or TCP, for instance.
- *server side:*
 1. Create a socket with the `socket()` system call
 2. Bind the socket to an address using the `bind()` system call. For a server socket on the Internet, an address consists of a port number and the host address.
 3. Listen for connections with the `listen()` system call
 4. Accept a connection with the `accept()` system call. This call typically blocks until a client connects with the server.
 5. Send and receive data via UDP or TCP, for instance.

Sockets and ports



User Datagram Protocol (UDP)

- Messages are sent without acknowledgement of receipt or retry
- **Message size:** the receiver decides on the size of the buffer for receiving purposes. Longer messages are truncated
- **Blocking:** usually, non-blocking send and blocking receive (eventually with timeout)
- **Receive from any:** this is the default, but a fixed sender socket is possible

User Datagram Protocol (UDP)

- **Failure model:**
 - *Omission failures:* datagrams may be dropped because there is no listening process on a port, or because the buffer is full
 - *Ordering failures:* there is no guarantee of order of delivery
- UDP is less reliable than TCP, but as it has less overhead it may be convenient to use. For example, DNS uses UDP

Java UDP API

- DatagramPacket:
 <message, length, internet address, port>
- DatagramSocket:
 To create and manages socket connections for sending
 UDP datagrams

Transfer Control Protocol (TCP)

- Gives the abstraction of a streaming connection. This includes hiding of:
 - message size
 - lost messages (if no ack, then resend)
 - full buffers (flow control to control consume speed)
 - unordered messages (numbering scheme of packets)
 - destination (connection oriented)
- Failure model:
 - Checksum for integrity checking
 - Timeouts for validity checking (problem: no way to differentiate between network problems and connection flush of one of the processes)

Java TCP API

- ServerSocket (for server listening on a port)
- Socket (for connecting to remote processes)

External data representation

- The transport layer is only concerned with the reliable transmission of sequences of bytes, but what about the type of information of transmitted data?
 1. The communicating processes agree on an external format (e.g., ASCII characters)
 2. The sender informs the receiver of the format together with the data

Marshalling

- **Marshalling:** the process of encoding a set of data items into a coherent form for the purpose of transmission
- **Unmarshaling:** the corresponding decoding process

Three examples

- Corba common's data representation
- Java's serialization
- Extensible Markup Language (XML)
 - middleware or application marshalling/unmarshalling?
 - transformed into binary or textual?
 - self-describing or external references? (namespaces)
- other examples: protocol buffers, JSON

Corba Common Data Representation

- data types for invocations
- 15 primitive types
- constructed types (ordered sequence)

Figure 4.7

CORBA CDR for constructed types

<i>Type</i>	<i>Representation</i>
<i>sequence</i>	length (unsigned long) followed by elements in order
<i>string</i>	length (unsigned long) followed by characters in order (can also can have wide characters)
<i>array</i>	array elements in order (no length specified because it is fixed)
<i>struct</i>	in the order of declaration of the components
<i>enumerated</i>	unsigned long (the values are specified by the order declared)
<i>union</i>	.type tag followed by the selected member

Figure 4.8
CORBA CDR message

<i>index in sequence of bytes</i>	<i>4 bytes</i>	<i>notes on representation</i>
0–3	5	<i>length of string</i>
4–7	"Smit"	'Smith'
8–11	"h____"	
12–15	6	<i>length of string</i>
16–19	"Lond"	'London'
20–23	"on____"	
24–27	1984	<i>unsigned long</i>

The flattened form represents a *Person* struct with value: {'Smith', 'London', 1984}

Figure 4.10 XML definition of the Person structure

```
<person id="123456789">  
    <name>Smith</name>  
    <place>London</place>  
    <year>1984</year>  
    <!-- a comment -->  
</person >
```

Figure 4.11 Illustration of the use of a namespace in the *Person* structure

```
<person pers:id="123456789" xmlns:pers = "http://www.cdk5.net/person">  
  <pers:name> Smith </pers:name>  
  <pers:place> London </pers:place >  
  <pers:year> 1984 </pers:year>  
</person>
```

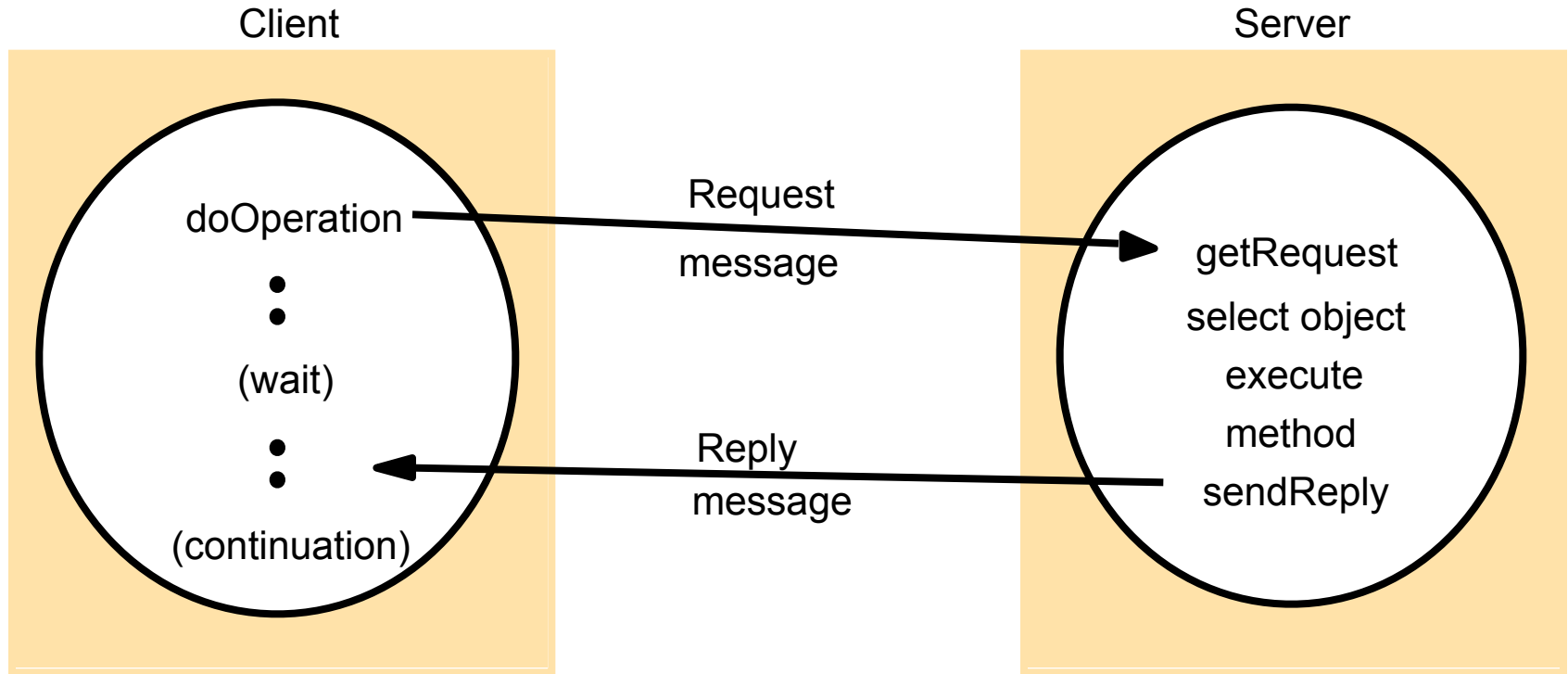
Figure 4.12 An XML schema for the *Person* structure

```
<xsd:schema xmlns:xsd = URL of XML schema definitions    >
  <xsd:element name= "person" type = "personType" />
    <xsd:complexType name="personType">
      <xsd:sequence>
        <xsd:element name = "name" type="xs:string"/>
        <xsd:element name = "place" type="xs:string"/>
        <xsd:element name = "year" type="xs:positiveInteger"/>
      </xsd:sequence>
      <xsd:attribute name= "id" type = "xs:positiveInteger"/>
    </xsd:complexType>
  </xsd:schema>
```

Client-Server communication

- Typical example of interprocess communication, based on the request-reply protocol
- Typical facts:
 - Acknowledgements are redundant (the reply is an ack)
 - Connection implies an extra pair of messages
 - Flow control is redundant

Request-reply communication



Operations of a request-reply protocol

public byte[] doOperation (RemoteObjectRef o, int methodId, byte[] arguments)

sends a request message to the remote object and returns the reply.

The arguments specify the remote object, the method to be invoked and the arguments of that method.

public byte[] getRequest ();

acquires a client request via the server port.

public void sendReply (byte[] reply, InetAddress clientHost, int clientPort);

sends the reply message reply to the client at its Internet address and port.

A request-reply message structure

messageType	<i>int (0=Request, 1= Reply)</i>
requestId	<i>int</i>
objectReference	<i>RemoteObjectRef</i>
methodId	<i>int or Method</i>
arguments	<i>array of bytes</i>

Request-reply failure model

- Omission failure and ordering failure
 - Timeout of the *doOperation* or of the *send*
 - Discarding duplicate requests
 - Loosing reply messages
 - History to cope with duplicate requests (when the reply might have been lost)

RPC exchange protocols

<i>Name</i>	<i>Messages sent by</i>		
	<i>Client</i>	<i>Server</i>	<i>Client</i>
R	<i>Request</i>		
RR	<i>Request</i>	<i>Reply</i>	
RRA	<i>Request</i>	<i>Reply</i>	<i>Acknowledge reply</i>

An example of request-reply: HTTP

- A protocol implemented over TCP
- Address of resource= URL
 - http://<host address>:<port>/resource
- Content negotiation (external data formats)
- Authentication is supported
- Persistent connections are supported (a request of a webpage may consist of several individual request-replies of resources)

HTTP request message

<i>method</i>	<i>http URL or pathname</i>	<i>HTTP version</i>	<i>headers</i>	<i>message body</i>
GET	//www.nike.com:8080/a.htm	HTTP/ 1.1		

POST netaddress,
 port,
HEAD resource identifier

PUT

DELETE

OPTIONS

TRACE

HTTP reply message

<i>HTTP version</i>	<i>status code</i>	<i>reason</i>	<i>headers</i>	<i>message body</i>
HTTP/1.1	200	OK		resource data

Group communication

- Multicast transmission: a message sent to a specified group of recipients
- Examples:
 - Fault tolerance based on replicated services (requests go to all servers)
 - Spontaneous networking (all members of the network receive messages)
 - Better performance through replicated data (the updated data goes to all storing the data)
 - Event notification

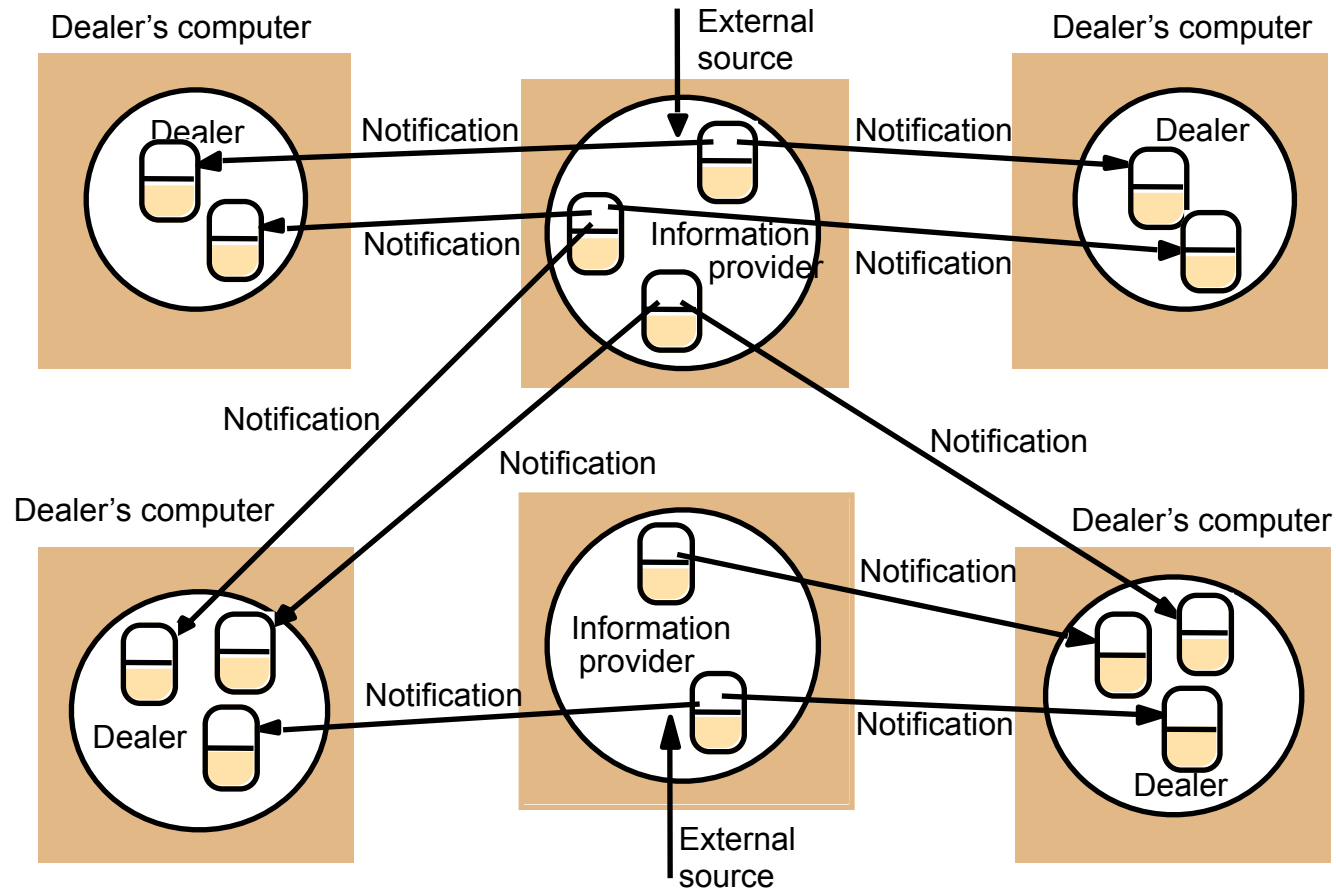
IP multicast

- Multicast group specified by a class D address
- Available only as UDP
- Hosts may join and leave multicast address dynamically
- Multicast routers allow for multicast messages over the whole Internet, not only local networks. Time To Live (TTL) may specify the max router hops
- Java: *multicastSocket*

Events and Notifications

- Publish/subscribe paradigm
 - Event types
 - Notifications
 - Registration groups
- Typically for systems which are:
 - Heterogenous
 - Asynchronous
 - Varying scale

Figure 5.9
Dealing room system



Event Notifications

- Roles:
 - Observer, object of interest, event, notification, subscriber, publisher
- Delivery semantics:
 - Maybe, e.g., via IP multicast
 - At-least-once, via request-reply multicast
 - Other options
- Observer
 - Forwarding
 - Filtering of notifications
 - Patterns of events
 - Notification mailboxes
- Jini implements an event notification mechanism

Figure 5.10
Architecture for distributed event notification

